

5.1 C Programming

C is a general-purpose, procedural computer programming language supporting structured programming, and recursion, with a static type system. C is the most widely used computer language. It keeps fluctuating at number one scale of popularity along with Java programming language and Python scripting.

Interesting Information:

C programming is a general-purpose, procedural, imperative computer programming language developed in 1972 by Dennis M. Ritchie at the Bell Telephone Laboratories to develop the UNIX operating system.

Why to Learn C Programming?

C programming language is a must for students and working professionals to become a software engineer especially when they are working in software development domain.

Some key advantages of learning C Programming are:

- Easy to learn.
- Structured language.
- It produces efficient programs.
- It can handle low-level activities.
- It can be compiled on a variety of computer platforms.

Facts about C

- C was invented to write an operating system called UNIX.
- C is a successor of B language which was introduced around the early 1970s.
- The language was formalized in 1988 by the American National Standard Institute (ANSI).
- The UNIX OS was totally written in C.
- Today C is the most widely used and popular System Programming Language.
- Most of the state-of-the-art software have been implemented using C.

5.2 Structure of a C Program

Hello World Example

A C program basically consists of the following parts:

- Preprocessor Commands
- Statements & Expressions
- Main Function

As an example, see the following program to print “Hello World”:

```
#include<stdio.h>    // Preprocessor Commands

int main(){          // Main Function
/* my first program in C */
    printf("Hello, World! \n"); // Statements

return0;
}
```

- The first line of the program *#include <stdio.h>* is a preprocessor command, which tells a C compiler to include *stdio.h* file before going to actual compilation.
- The next line *int main()* is the main function where the program execution begins.
- The next line */*...*/* will be ignored by the compiler and it has been put to add additional comments in the program. Such lines are called comments in the program.
- The next line *printf(...)* is another function available in C which causes the message "Hello, World!" to be displayed on the screen.
- The next line **return 0;** terminates the *main()* function and returns the value 0.

5.3 Life Cycle of a C Program

A C program goes through the following stages before execution:

- **Preprocessing** - Using a Preprocessor program to convert C source code in expanded source code. "*#includes*" and "*#defines*" statements will be processed and replaced with the source codes in this step.

- **Compilation** - Using a Compiler program to convert C expanded source to assembly source code.
- **Assembly** - Using an Assembler program to convert assembly source code to object code.
- **Linking** - Using a Linker program to convert object code to executable code. Multiple units of object codes are linked together in this step.
- **Loading** - Using a Loader program to load the executable code into CPU for execution.

5.4 Setting up Programming Environment

There are plenty of Integrated Development Environments (IDEs) available for C language. We will only focus of Microsoft Visual Studio in this book.

Getting Started with Microsoft Visual Studio

1. Download the setup from official visual studio website.
2. Install visual studio on your computer.
3. Open the application
4. Click new project
5. Select Empty Project and name the project as shown in Figure 5.1.

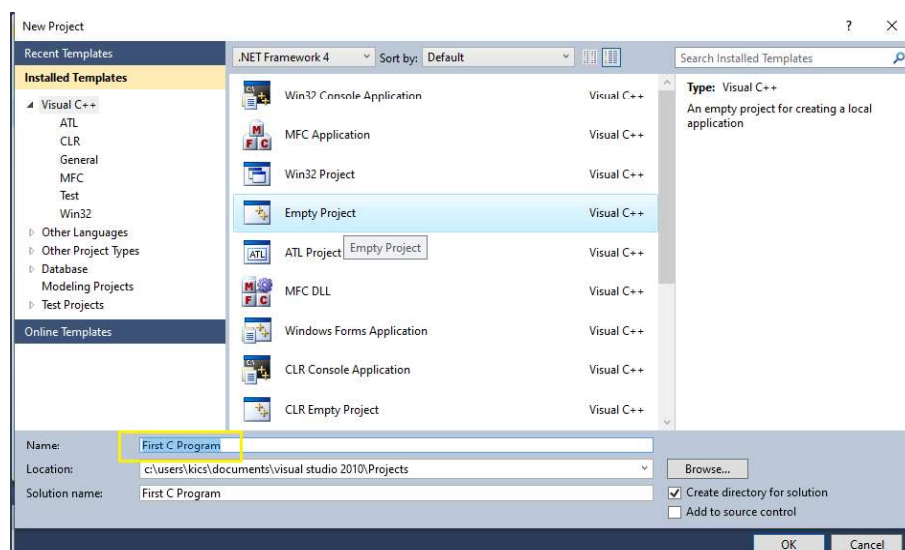


Fig. 5.1 Creating a new project in visual studio

6. Click on add new source (as shown in Figure 5.2).

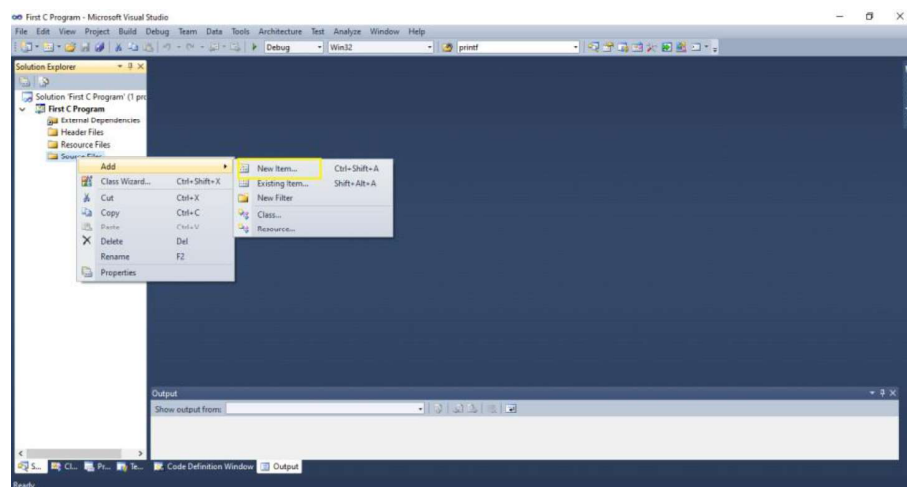


Fig. 5.2 Creating a new Resource File in Visual Studio

7. Select C++ file and name this file (see Figure 5.3).

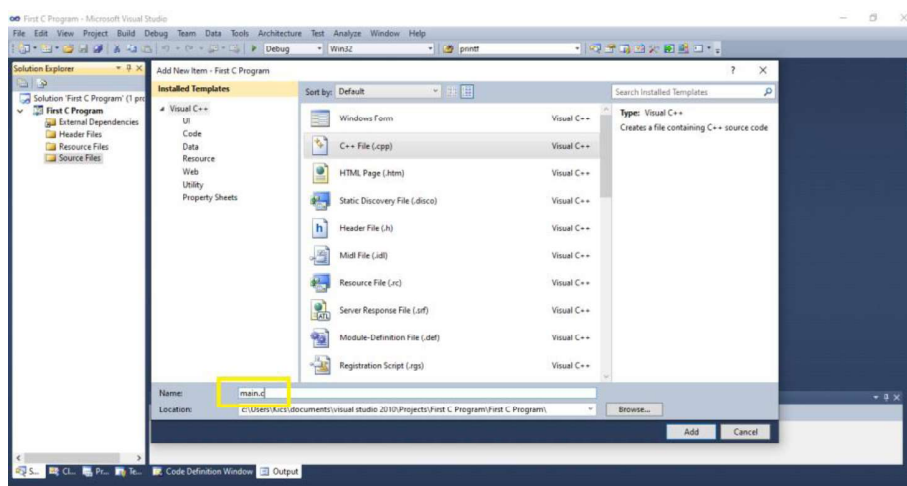


Fig. 5.3 Selecting File Type in Visual Studio

8. Write this code (Figure 5.4) in your source file.

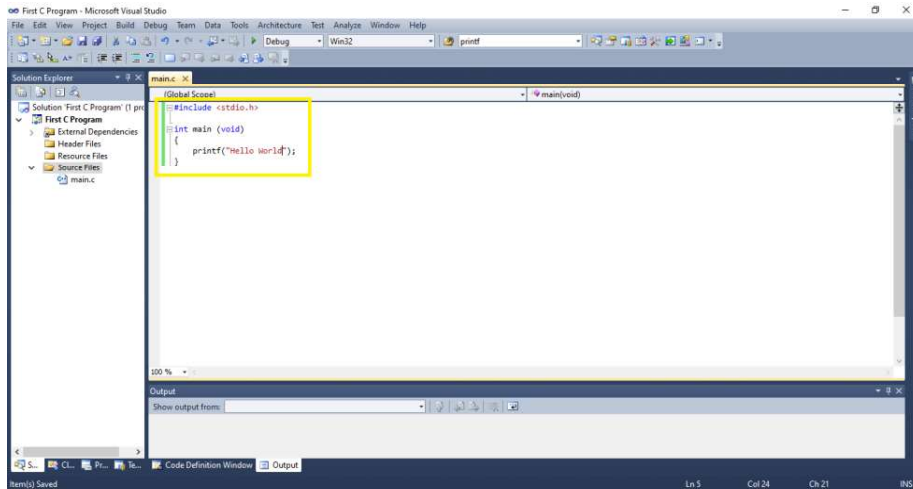


Fig. 5.4 C Program to print Hello World

9. Compile/build the code following the procedure shown in Figure 5.5.

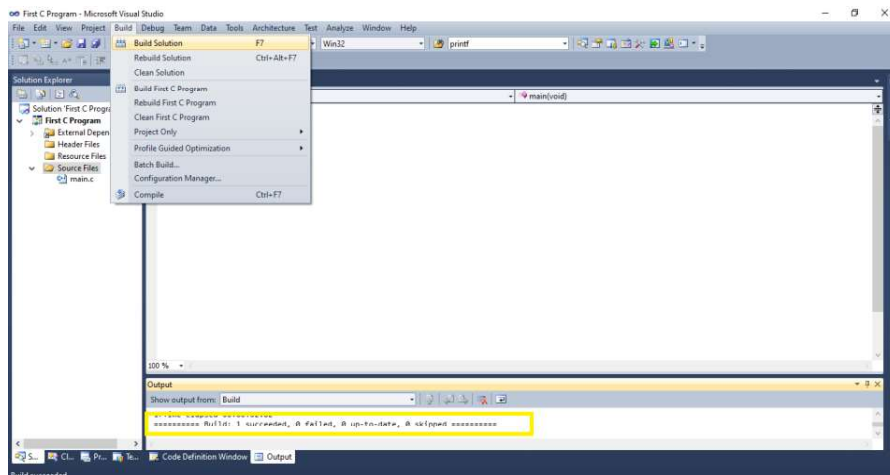


Fig. 5.5 Compiling a C Program in Visual Studio

10. Debug the Program following the procedure in Figure 5.6.

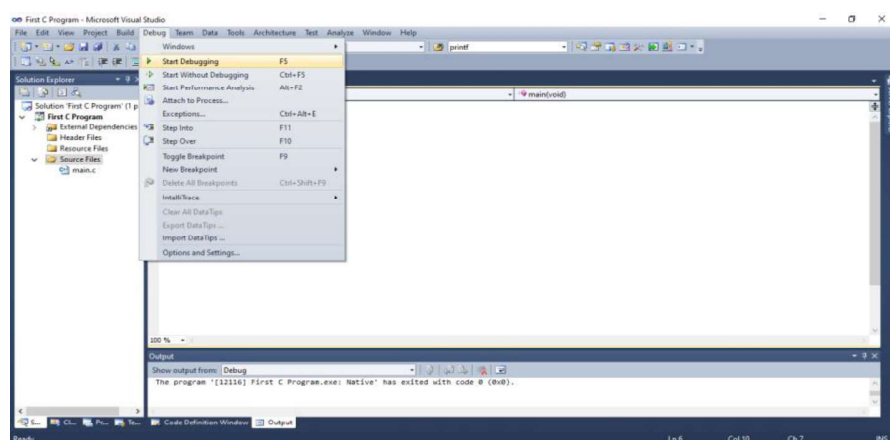


Fig. 5.6 Debugging a C Program in Visual Studio

11. See the output as depicted in Figure 5.7.

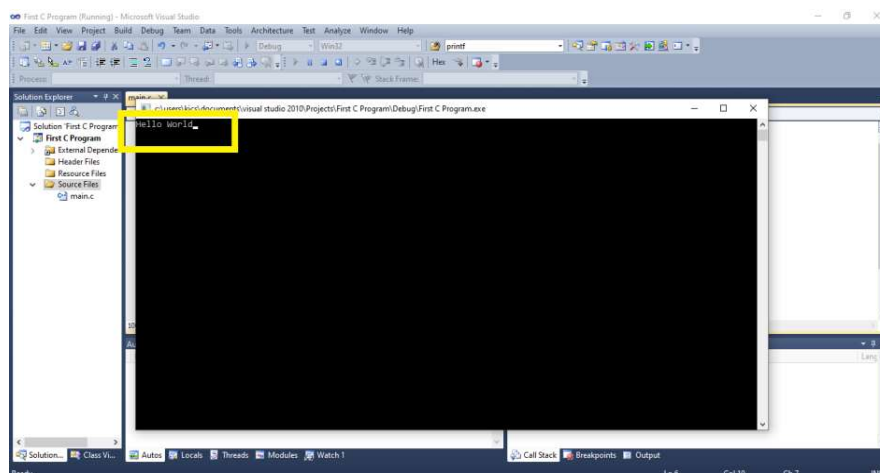


Fig. 5.7 Output of Hello World C Program

Step-in debugging in Microsoft Visual Studio

One very useful feature of Microsoft Visual Studio is the step wise debugging of code. You can run your code line by line to find out the reason for errors rather than running the code at once. Follow Figure 5.8 to perform step-wise debugging.

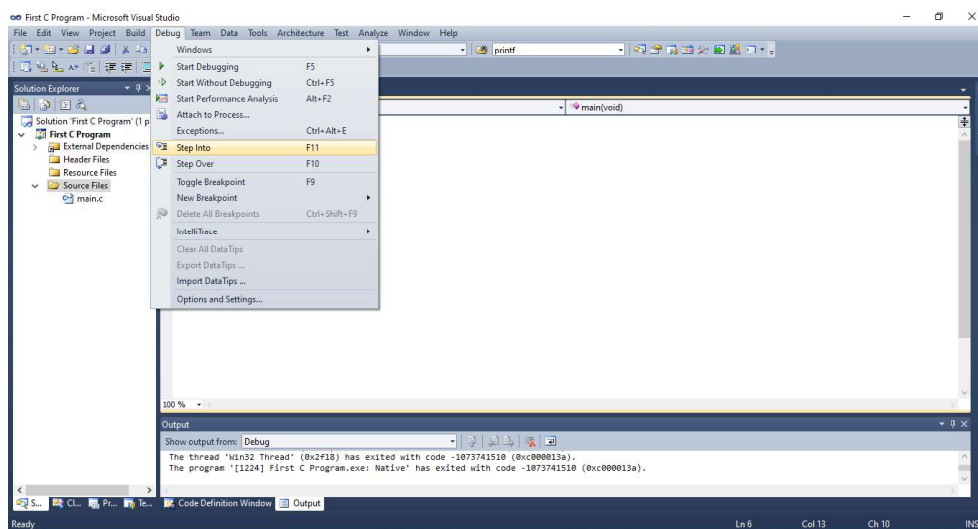


Fig. 5.7 Step-wise Debugging in Visual Studio

5.5 Input/Output Streams in C

Input means feeding some data into a program. An input can be given in the form of a file or from the command line. C programming provides a set of built-in functions to read the given input and feed it to the program as per requirement.

Output means displaying some data on screen, printer, or in any file. C programming provides a set of built-in functions to output the data on the computer screen as well as to save it in text or binary files.

Scanf() and Printf() Functions

The **int scanf(const char *format, ...)** function reads the input from the standard input stream **stdin** and scans that input according to the **format** provided.

The **int printf(const char *format, ...)** function writes the output to the standard output stream **stdout** and produces the output according to the format provided.

The **format** can be a simple constant string, but you can specify %s, %d, %c, %f, etc., to print or read strings, integer, character or float respectively. There are many other formatting options available which can be used based on requirements. We provide a simple example below to understand the concepts better:

```
#include<stdio.h>
int main(){
    char str[100];
    int i;

    printf("Enter a value :");
    scanf("%s %d", str,&i);

    printf("\nYou entered: %s %d ", str, i);

    return 0;
}
```

When the above code is compiled and executed, it waits for the user to input some text. When the user enters a text and presses enter, then the program proceeds and reads the input and displays it as follows:

Enter a value: seven 7

You entered: seven 7

Here, it should be noted that scanf() expects input in the same format as you provided %s and %d, which means you have to provide valid inputs like "string integer". If you provide "string string" or "integer integer", then it will be assumed as wrong input.

Secondly, while reading a string, `scanf()` stops reading as soon as it encounters a space, so "this is test" are three strings for `scanf()`.

Format Specifiers

Format specifiers are used in C for input and output purposes. Using this concept, the compiler can understand that what type of data is in a variable during taking input using the `scanf()` function and printing using `printf()` function. Here is a list of format specifiers.

Format Specifier	Type
%c	Character
%d	Signed integer
%f	Float values
%hi	Signed integer (short)
%hu	Unsigned Integer (short)
%i	Unsigned integer
%l or %ld or %li	Long
%lf	Double
%Lf	Long double
%lu	Unsigned int or unsigned long
%lli or %lld	Long long
%llu	Unsigned long long
%o	Octal representation
%p	Pointer
%s	String
%u	Unsigned int

Format Specifier	Type
%x or %X	Hexadecimal representation

These are the basic format specifiers. We can add some other parts with the format specifiers:

- A minus symbol (-) sign tells left alignment
- A number after % specifies the minimum field width. If string is less than the width, it will be filled with spaces
- A period (.) is used to separate field width and precision

Example:

```
#include<stdio.h>

int main(){

    char ch ='B';

    printf("%c\n", ch); //printing character data
    //print decimal or integer data with d and i

    int x =45, y =90;

    printf("%d\n", x);
    printf("%i\n", y);

    float f =12.67;

    printf("%f\n", f); //print float value
    printf("%e\n", f); //print in scientific notation

    int a =67;

    printf("%o\n", a); //print in octal format
    printf("%x\n", a); //print in hex format

    char str []="Hello World";

    printf("%s\n", str);

    printf("%20s\n", str); //shift to the right 20 characters
    including the string
```

```

printf("%-20s\n", str); //left align

printf("%20.5s\n", str); //shift to the right 20 characters
including the string, and print string up to 5 character

printf("%-20.5s\n", str); //left align and print string up to 5
character
}

```

Output

```

B
45
90
12.670000
1.267000e+001
103
43
Hello World
Hello World
Hello World
Hello
Hello

```

We can use these format specifiers for the scanf() function in the same manner.

5.5.1 Data Types

Data types in C refer to an extensive system used for declaring variables or functions of different types. The type of a variable determines how much space it occupies in storage and how the bit pattern stored is interpreted.

Integer Types

The following table provides the details of standard integer types with their storage sizes and value ranges:

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255

signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes or (4bytes for 32-bit OS)	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

To get the exact size of a type or a variable on a particular platform, you can use the **sizeof** operator. The expressions *sizeof(type)* yields the storage size of the object or type in bytes. Given below is an example to get the size of various type on a machine using different constant defined in limits.h header file:

```
#include<stdio.h>
#include<stdlib.h>
#include<limits.h>
#include<float.h>
int main(int argc, char** argv) {
    printf("CHAR_BIT      :   %d\n", CHAR_BIT);
    printf("CHAR_MAX       :   %d\n", CHAR_MAX);
    printf("CHAR_MIN       :   %d\n", CHAR_MIN);
    printf("INT_MAX        :   %d\n", INT_MAX);
    printf("INT_MIN        :   %d\n", INT_MIN);
    printf("LONG_MAX       :   %ld\n", (long) LONG_MAX);
    printf("LONG_MIN       :   %ld\n", (long) LONG_MIN);
    printf("SCHAR_MAX      :   %d\n", SCHAR_MAX);
    printf("SCHAR_MIN      :   %d\n", SCHAR_MIN);
    printf("SHRT_MAX       :   %d\n", SHRT_MAX);
    printf("SHRT_MIN       :   %d\n", SHRT_MIN);
    printf("UCHAR_MAX      :   %d\n", UCHAR_MAX);
    printf("UINT_MAX       :   %u\n", (unsignedint) UINT_MAX);
    printf("ULONG_MAX      :   %lu\n", (unsignedlong) ULONG_MAX);
    printf("USHRT_MAX      :   %d\n", (unsignedshort) USHRT_MAX);
return 0;
}
```

When you compile and execute the above program, it produces the following result (specifically for Linux):

```

CHAR_BIT      :    8
CHAR_MAX      :   127
CHAR_MIN      :  -128
INT_MAX       :  2147483647
INT_MIN       : -2147483648
LONG_MAX      :  9223372036854775807
LONG_MIN      : -9223372036854775808
SCHAR_MAX     :   127
SCHAR_MIN     :  -128
SHRT_MAX      :   32767
SHRT_MIN      :  -32768
UCHAR_MAX     :   255
UINT_MAX      :  4294967295
ULONG_MAX     : 18446744073709551615
USHRT_MAX     :   65535

```

Floating-Point Types

The following table provides the details of standard floating-point types with storage sizes and value ranges and their precision:

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places

The header file *float.h* defines macros that allow you to use these values and other details about the binary representation of real numbers in your programs. The following example prints the storage space taken by a float type and its range values:

```

#include<stdio.h>
#include<stdlib.h>
#include<limits.h>
#include<float.h>

int main(int argc, char** argv) {

    printf("Storage size for float : %d \n", sizeof(float));
    printf("FLT_MAX      :    %g\n", (float) FLT_MAX);
    printf("FLT_MIN      :    %g\n", (float) FLT_MIN);
    printf("-FLT_MAX     :    %g\n", (float)-FLT_MAX);
    printf("-FLT_MIN     :    %g\n", (float)-FLT_MIN);
    printf("DBL_MAX      :    %g\n", (double) DBL_MAX);
    printf("DBL_MIN      :    %g\n", (double) DBL_MIN);
    printf("-DBL_MAX     :    %g\n", (double)-DBL_MAX);
    printf("Precision value: %d\n", FLT_DIG );
}

```

```
return 0;  
}
```

When you compile and execute the above program, it produces the following result on Linux:

```
Storage size for float : 4  
FLT_MAX    : 3.40282e+38  
FLT_MIN    : 1.17549e-38  
-FLT_MAX   : -3.40282e+38  
-FLT_MIN   : -1.17549e-38  
DBL_MAX    : 1.79769e+308  
DBL_MIN    : 2.22507e-308  
-DBL_MAX   : -1.79769e+308  
Precision value: 6
```

Void Type

The void type specifies that no value is available. It is used in the following kinds of situations:

Sr.No.	Types & Description
1	Function returns as void There are various functions in C which do not return any value or you can say they return void. A function with no return value has the return type as void. For example, void exit (int status);
2	Function arguments as void There are various functions in C which do not accept any parameter. A function with no parameter can accept a void. For example, int rand(void);

5.6 Arithmetic Operators in C

An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. C language is rich in built-in operators and provides the following types of operators –

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Miscellaneous Operators

Arithmetic Operators

The following table shows all the arithmetic operators supported by the C language. Assume variable **A** holds 10 and variable **B** holds 20 then:

Operator	Description	Example
+	Adds two operands.	$A + B = 30$
-	Subtracts second operand from the first.	$A - B = -10$
*	Multiplies both operands.	$A * B = 200$
/	Divides numerator by de-numerator.	$B / A = 2$
%	Modulus Operator and remainder of after an integer division.	$B \% A = 0$
++	Increment operator increases the integer value by one.	$A++ = 11$
--	Decrement operator decreases the integer value by one.	$A-- = 9$

Example

See the following example to understand all the arithmetic operators available in C:

```
#include<stdio.h>

main() {

    int a =21;
    int b =10;
    int c ;

    c = a + b;
    printf("Line 1 - Value of c is %d\n", c );

    c = a - b;
    printf("Line 2 - Value of c is %d\n", c );

    c = a * b;
    printf("Line 3 - Value of c is %d\n", c );

    c = a / b;
    printf("Line 4 - Value of c is %d\n", c );

    c = a % b;
    printf("Line 5 - Value of c is %d\n", c );

    c = a++;
    printf("Line 6 - Value of c is %d\n", c );

    c = a--;
    printf("Line 7 - Value of c is %d\n", c );
}
```

When you compile and execute the above program, it produces the following result –

```
Line 1 - Value of c is 31
Line 2 - Value of c is 11
Line 3 - Value of c is 210
Line 4 - Value of c is 2
Line 5 - Value of c is 1
Line 6 - Value of c is 21
Line 7 - Value of c is 22
```

Relational Operators

The following table shows all the relational operators supported by C. Assume variable **A** holds 10 and variable **B** holds 20 then:

Operator	Description	Example
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B) is not true
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand. If yes, then the condition becomes true.	(A <= B) is true.

Example

See the following example to understand all the relational operators available in C:

```
#include<stdio.h>

main() {

int a =21;
int b =10;
int c ;

if( a == b ){
    printf("Line 1 - a is equal to b\n");
}else{
    printf("Line 1 - a is not equal to b\n");
}

if( a < b ){
    printf("Line 2 - a is less than b\n");
}else{
    printf("Line 2 - a is not less than b\n");
}
```

```

if( a > b ){
    printf("Line 3 - a is greater than b\n");
}else{
    printf("Line 3 - a is not greater than b\n");
}

/* Lets change value of a and b */
a =5;
b =20;

if( a <= b ){
    printf("Line 4 - a is either less than or equal to b\n");
}

if( b >= a ){
    printf("Line 5 - b is either greater than or equal to b\n");
}

```

When you compile and execute the above program, it produces the following result:

```

Line 1 - a is not equal to b
Line 2 - a is not less than b
Line 3 - a is greater than b
Line 4 - a is either less than or equal to b
Line 5 - b is either greater than or equal to b

```

Logical Operators

The Following table shows all the logical operators supported by C language. Assume variable **A** holds 1 and variable **B** holds 0, then:

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false.
	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A && B) is true.

Example

See the following example to understand all the logical operators available in C:

```
#include<stdio.h>

main() {

int a =5;
int b =20;
int c ;

if( a && b ){
    printf("Line 1 - Condition is true\n");
}

if( a || b ){
    printf("Line 2 - Condition is true\n");
}

/* lets change the value of a and b */
a =0;
b =10;

if( a && b ){
    printf("Line 3 - Condition is true\n");
}else{
    printf("Line 3 - Condition is not true\n");
}

if(!(a && b)){
    printf("Line 4 - Condition is true\n");
}

}
```

When you compile and execute the above program, it produces the following result:

```
Line 1 - Condition is true
Line 2 - Condition is true
Line 3 - Condition is not true
Line 4 - Condition is true
```

Bitwise Operators

Bitwise operator works on bits and perform bit-by-bit operation. The truth tables for &, |, and ^ is as follows:

p	Q	p & q	p q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Assume A = 60 and B = 13 in binary format, they will be as follows:

A = 0011 1100

B = 0000 1101

A&B = 0000 1100

A|B = 0011 1101

A^B = 0011 0001

~A = 1100 0011

The following table lists the bitwise operators supported by C. Assume variable 'A' holds 60 and variable 'B' holds 13, then:

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) = 12, i.e., 0000 1100
	Binary OR Operator copies a bit if it exists in either operand.	(A B) = 61, i.e., 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) = 49, i.e., 0011 0001
~	Binary One's Complement Operator is unary and has the effect of 'flipping' bits.	(~A) = ~(60), i.e., -0111101

<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 = 240 i.e., 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 = 15 i.e., 0000 1111

Example

See the following example to understand all the bitwise operators available in C:

```
#include<stdio.h>

main() {

    unsignedint a =60;  /* 60 = 0011 1100 */
    unsignedint b =13;  /* 13 = 0000 1101 */
    int c =0;

    c = a & b; /* 12 = 0000 1100 */
    printf("Line 1 - Value of c is %d\n", c );

    c = a | b; /* 61 = 0011 1101 */
    printf("Line 2 - Value of c is %d\n", c );

    c = a ^ b; /* 49 = 0011 0001 */
    printf("Line 3 - Value of c is %d\n", c );

    c = ~a; /* -61 = 1100 0011 */
    printf("Line 4 - Value of c is %d\n", c );

    c = a <<2; /* 240 = 1111 0000 */
    printf("Line 5 - Value of c is %d\n", c );

    c = a >>2; /* 15 = 0000 1111 */
    printf("Line 6 - Value of c is %d\n", c );
}
```

When you compile and execute the above program, it produces the following result:

```
Line 1 - Value of c is 12
Line 2 - Value of c is 61
Line 3 - Value of c is 49
Line 4 - Value of c is -61
Line 5 - Value of c is 240
Line 6 - Value of c is 15
```